

Post-OCR Error Processing of Scanned Chess PGN Images Using Statistical Edit Distances and Branching

Erik Davtyan
Capstone with Chessify
American University of Armenia
Computer Science
erik_davtyan@edu.aua.am

Abstract

When trying to scan chess PGNs to digitize them, OCR errors can bring about significant diversions from the original game. There are more possible chess games than elementary particles in the universe - so the possibilities are almost endless. A single error close to the beginning of the game makes it hard to make localized-error corrections. However, using post-OCR processing tools in combination with clever back-tracking algorithms and statistical edit distances it is possible to reach high-fidelity results.

Keywords

OCR errors, OCR post-processing, post-OCR text correction, chess OCR, chess pgn digitization

Introduction

OCR techniques have been used extensively on some error-intolerant tasks like postal addressing and bank cheques. However, the technology is still not perfect. To make things easier, the systems usually use specifically made fonts which make it easier to understand scanned characters. However, in cases where there is a lot of text and no

special font, OCR techniques result in mistakes. For example, using Tesseract (version 4.1.1), there were 5088 wrongly scanned characters (555 lost, 263 wrongly added, and 4270 changed to a wrong character) out of a total of 324169 characters in the training set. So, if we try to digitally traverse over all the scanned moves, the character mistakes will result in illegal moves, thus we will not be able to recreate the game. Considering that there are on average 247.6 characters in a single game, the chance that there will be a character error in an average game is about 98%. So, to implement a robust PGN digitizer, it is important to build post-processing systems which will get rid of as many errors as possible.

Preparation

Chess Portable Game Notations (PGN) are a standard learning tool in chess books. In this analysis Standard Algebraic Notation (SAN) is used, in which the legal characters are Latin letters, numbers, a few algebraic symbols (+ for check, # for mate, etc.), dots and dashes. Using SAN makes it easy to use pre-trained OCR models like Tesseract, but in principle the post-OCR error processing can be done with other notation types (using the figure symbols themselves) and a

custom-trained OCR model. Pictures of PGN-s in SAN are not easy to find in bulk. So, to make the data collection process easier, I scraped all the PGN text files from <https://www.pgnmentor.com> (389505 games total). Then, using the Python PIL and chess libraries, I extracted all the games of Akopian from the text file and created training images from those games¹ (1313 games total). During this process I also created a “golden output” file in json format from the actual games, to then compare my analysis results to the actual games. As the dataset generation and the later analysis of it requires a lot of computing power, I decided to try my algorithm on these 1313 games first. Then I created a simple Tesseract configuration -

```
tessedit_char_whitelist=01234  
56789.abcdefghijklmnopqrstuvwxyz\
```

, to include only the characters which can be in a PGN file. Javascript's chess.js library gives an API that is very flexible for my use-case. However, the performance of javascript's Tesseract port is slow, so I used the above mentioned configuration of Tesseract in a python script which takes as argument the path of the image to OCR, and writes the scanned image to the stdout. This way, I could spawn a process in javascript to execute the more performant python script, and analyze the result using the more convenient chess.js library. Now, we have

¹

<https://nicholastsmith.wordpress.com/2017/10/14/deep-learning-ocr-using-tensorflow-and-python/>

1313 chess games in images, and their string representations in a golden output file. We also have scripts to scan an image to compare it to the golden output. I also created a testing set from all the games of Alburto (776 games). The function which splits moves into an array uses a Regular Expression. Unfortunately, in a few cases, the function is unable to split a move (4 in 1313 of the training set, 0 in the testing set). So, for the purpose of this analysis I am going to ignore those games. However, if a machine learning algorithm is trained to split the moves and deal with those 4 games also, the rest of the analysis will work perfectly for those games as well. Now, we can start the analysis.

Initial analysis

At first I ran a quick analysis (firstAnalysis.json) to see where I stand before any post-processing efforts. I wrote a move difference calculator, which is a simple array item comparing function that returns how many of the elements are not equal. Then I ran both functions on all move-splitted games from the training set to get the average difference between scanned and real games. The average difference is ~5.04. Here is the full statistical summary of the results:

Train data:

Mean: 5.041127189642041,

First quartile: 1,

Median: 3,

Third quartile: 5,

Min: 0,

Max: 97,
Sum: 6619
of games with 0 errors: 130
of games with 1 errors: 224

Adding simple post-processing of split moves, like removing illegal characters from SAN (for example - '.', and replace 0 with O), already yields better results (secondAnalysis.json). Instead of hard coding changin 0 with O we can use statistical edit distances, which is mentioned later in the paper.

Train data:
Mean: 3.8552932216298554,
First quartile: 1,
Median: 3,
Third quartile: 5,
Min: 0,
Max: 77,
Sum: 5062
of games with 0 errors: 166
of games with 1 errors: 243

I continued searching for common errors that happen due to OCR. It turned out changing the split moves function to be more strict about the splitting increases fidelity even further, as a lot of them had a . scanned instead of a space, which was being mistaken for a separate move. This happening at the start of the game was changing the ordering of all following moves, effectively giving an error for almost every move. Let's try the analysis function again on the entire dataset, after making the splitting regular expression stricter.

Train data:
Mean: 3.431835491241432,
First quartile: 1,
Median: 3,
Third quartile: 5,
Min: 0,
Max: 27,
Sum: 4506,
Perfect scans: 173
1 mistake scans: 242

Test data:
Mean: 4.119845360824742,
First quartile: 1,
Median: 3,
Third quartile: 6,
Min: 0,
Max: 116,
Sum: 3197
of games with 0 errors: 77
of games with 1 errors: 126
of games with 2 errors: 118
of games with 3 errors: 105

While analysing the data I noticed a lot of the scans mistook the letter 'c' for 'e'. Running an analysis on the entire dataset (single moves) yields 2977 cases of c being mistook for e (total e-s 21824).

Edit distances

To solve the problem of moves being scanned as different ones, I used the Levenshtenian distance algorithm. Basically, it returns a number of how many steps are necessary to turn one string into another. For example - levenshtein(e4, c4) = 1, as we

only need to substitute e with c to get the string “c4”.

I implemented a function which traverses all scanned moves within a game. For all legal moves, the function simply adds that move to the history of the game. When an illegal move is reached (mistake in the scan) the function gets all the legal moves from the position, and finds the one that has the least Levenshtenian distance from the scanned string. It then plays that move, and continues traversing.

However, this significantly increased the number of mistakes in the games. The problem was that many of the scanned moves were equidistant from 2 or more legal ones (*equality problem*). For example - if the wrongly scanned move is “e4” and the legal moves include both “c4” and “d4”, the algorithm has no way of knowing which one was actually “closer”, as both are 1 Levenshtenian distance away. After guessing the “closer” move wrong, the algorithm continued traversing from an already mistaken position, thereby returning the wrong results for every move after that one. So, using the recommendations from the paper “Deep Statistical Analysis of OCR Errors for Effective Post-OCR Processing”², I decided to make a statistical version of the Levenshtenian distance algorithm.

Statistical Edit Distances

2

https://www.researchgate.net/publication/335074056_Deep_Statistical_Analysis_of_OCR_Errors_for_Effective_Post-OCR_Processing

To make the calculations of edit distances more effective, I needed to reach the inequality

$\text{editDistance}('e4', 'c4') < \text{editDistance}('e4', 'd4')$,

as it is evident from the data that ‘c’ is more likely to get scanned as ‘e’. To get this inequality, I implemented the Wagner–Fischer algorithm (a dynamic algorithm for finding edit distances)³, but instead of adding 1 in the case of edits, I added specific costs for each character, so that the substitution cost from ‘e’ to ‘c’ would be lower than that of from ‘e’ to ‘d’.

To get the cost coefficients, I implemented another variation of the Wagner-Fischer algorithm, which instead of returning distances returns the actual edits necessary to get from the source string to the target one. For example,

```
edit('e4', 'c4') => [{ type:
'Substitute', from: 'e', to:
'c' }]
```

I then ran this algorithm over all the moves from the training games, and recorded the results (coefficients.json).

The results have the format of a dictionary with the keys and values:

Correct - a dictionary of the count characters correctly mapped to themselves (e.g. the character ‘2’ was correctly mapped to ‘2’ in 8873 cases,

Lost - a dictionary of the count of characters lost during scanning (e.g. the character ‘1’ was lost in 139)

³ <https://dl.acm.org/doi/10.1145/321796.321811>

Added - a dictionary of the count of characters wrongly added in during scanning (e.g. the character '0' was wrongly added in 3 cases)

Changed - a dictionary of the count of characters wrongly substituted for other characters (e.g. the character 'c' was wrongly scanned as 'e' in 2887 cases).

Using the counts above, I was able to calculate the probability of deletions, insertions, and substitutions of individual characters, and created a statistical model from those probabilities (model.json). The model has the same shape as the coefficients, but instead of counts of the events it has the probabilities of the events. Lastly, I used these probabilities to create the cost of edits using the function:

```
cost = (1 - P(mistake)) ^ 2
```

Raising it to the power of 2 increased the accuracy of the function by penalizing the lower probability mistakes even more. Here are the results.

Run on test data

```
Mean: 9.082474226804123,  
First quartile: 0,  
Median: 1,  
Third quartile: 9.25,  
Min: 0,  
Max: 136,  
Sum: 7048  
# of games with 0 errors: 379
```

To ignore the games where the traversing algorithm makes a mistake closer to the

start of the game and accumulates a lot of errors, I added a total distance threshold (*confidence coefficient*) of 10 distance. So, if the sum of all distances from scanned strings to legal move strings (*total distance of a game*) exceeds 10, I return the original scanned moves (which then can be manually corrected by users). Here are the results.

```
Mean: 2.8891752577319587,  
First quartile: 0,  
Median: 1,  
Third quartile: 5,  
Min: 0,  
Max: 116,  
Sum: 2242  
# of games with 0 errors: 377
```

While the number of games with 0 errors dropped by 2 (due to reduced confidence coefficient of the traversing function), the sum of all errors went down from 7048 to 2242.

Branching

Even after implementing the statistical edit distances, the *equality problem* still came up. The problem was that some events (e.g. '4' being scanned as 'e', or '4' being scanned as 'a') both had the same probability, as they happened the same amount of times in the training data. Even if we get larger training data and minimize equalities of probabilities, close probabilities still do not give us enough confidence. It is hard to say if the scanned string '45' was actually 'e5' or 'a5' in the real game. Thus, I implemented branching of games.

When the traversing algorithm reaches an illegal move string, which is epsilon close (*distance coefficient* = 0.01) to n different legal move strings -

$|\text{distance}(\text{moveStr}, \text{legalMove1}) - \text{distance}(\text{moveStr}, \text{legalMove2})| < \text{epsilon}$, the game gets branched out into n different games, and the traversing continues on all of them. After traversing every move, the game with the least *total distance* is selected from all branched games. Then, all games which are more than epsilon distance away from the closest game get discarded. This way the seemingly exponential branching algorithm in practice branches very few times, and very quickly discards most of the branches. Adding one more break condition for a large number of branched games (*max branching coefficient*) boosts up the performance to faster than a second per game. Running this algorithm on a game takes 1.08 second on average (on a 2.8 GHz Intel Core i7), with most games taking as little as 100 milliseconds. However, some games can take up to 10 seconds to traverse with branching. It is also possible to speed up the algorithm by having a low *max branching coefficient* = 40. That decreases the average time to 0.96 seconds, while increasing the mean of errors only by 0.05. This is in part due to an expensive implementation of the function which copies a ChessInstance object. However, only 1 game from my 776 test games took that long to scan, so those games can be discarded and fixed by user input.

Running this algorithm on the test data with different *confidence coefficients* yields these results.

Confidence coefficient: 15
Mean: 2.5876288659793816,
First quartile: 0,
Median: 0,
Third quartile: 4,
Min: 0,
Max: 116,
Sum: 2008
of games with 0 errors: 420
of games with 1 errors: 58
of games with 2 errors: 51
of games with 3 errors: 47

Confidence coefficient: 13
Mean: 2.51159793814433,
First quartile: 0,
Median: 0,
Third quartile: 4,
Min: 0,
Max: 116,
Sum: 1949
of games with 0 errors: 419
of games with 1 errors: 58
of games with 2 errors: 52
of games with 3 errors: 50

Confidence coefficient: 10
Mean: 2.4342783505154637,
First quartile: 0,
Median: 0,
Third quartile: 3,
Min: 0,
Max: 116,
Sum: 1889
of games with 0 errors: 418
of games with 1 errors: 59
of games with 2 errors: 53
of games with 3 errors: 53

of games with 4 errors: 42
of games with 5 errors: 35

As expected, increasing the *confidence coefficient* increases the number of games with 0 errors, but also increases the mean of errors in all games. This is because the traversing function “gives up” later, therefore more confidently identifying the correct moves in games with higher *total distances*. However, due to the same confidence, it blindly continues in many of the games where it already made a mistake, therefore reaching a larger number of mean errors. In practice, a *confidence coefficient* of 10 seems like the best choice. Using that coefficient gives us 53.8% games with 0 errors, 75.12% of games with less than 4 errors, and 85.05% games with less than 5 errors.

Conclusion

As we can see from the statistical summaries of the results, using statistical edit distances can dramatically reduce the number of errors in an error-intolerant OCR post-processing. Coupling this with branching of possibilities, it is possible to reduce the number of total errors by 40.09% (from 3197 to 1889), and increase the number of 0 error games 5.4 times (from 77 to 418). Using this technique it is possible to reach high fidelity results during post-processing game notations with well-defined legal moves similar to chess - like Go and Checkers.

References

- Nguyen, Thi-Tuyet-Hai & Jatowt, Adam & Coustaty, Mickaël & Nguyen, Nhu-Van & Doucet, Antoine. (2019). Deep Statistical Analysis of OCR Errors for Effective Post-OCR Processing. 29-38. 10.1109/JCDL.2019.00015. Retrieved from https://www.researchgate.net/publication/335074056_Deep_Statistical_Analysis_of_OCR_Errors_for_Effective_Post-OCR_Processing
- Nicholas T Smith. (2017). Deep Learning OCR using TensorFlow and Python. Retrieved from <https://nicholastsmith.wordpress.com/2017/10/14/deep-learning-ocr-using-tensorflow-and-python/>
- Wagner, Robert A.; Fischer, Michael J. (1974). "The String-to-String Correction Problem". *Journal of the ACM*. **21** (1): 168–173. doi:[10.1145/321796.321811](https://doi.org/10.1145/321796.321811)
- Dataset: <https://www.pgnmentor.com/files.html>